

Serverless Framework

Multi-Cloud Serverless — From concept to production in minutes

AWS Lambda

Azure Functions

Google Cloud

Open Source

Agenda

01



What is Serverless?

The paradigm & why it matters

02



The Serverless Framework

Architecture, YAML & core concepts

03



Multi-Cloud Support

AWS, Azure, GCP with one toolset

04



Live Demo & CLI Walkthrough

Deploy a function from scratch

05



Community & Comparisons

Ecosystem, pros/cons, alternatives

06



Resources & Key Takeaways

Where to go from here

What is Serverless?

"Serverless doesn't mean no servers — it means you stop thinking about them."



Traditional VM

- Always-on, pay 24/7
- Manual scaling & patching
- Full OS responsibility
- Minutes to provision



Containers / K8s

- Cluster running 24/7
- You own orchestration
- Faster than VMs
- Complex to operate



Serverless

- Scales to zero automatically
- Pay per ms of execution
- No infra to manage
- Event-driven by design

Why Serverless? The Business Case



Cost Efficiency

Zero idle cost. Pay only for actual compute time — often 10-90% cheaper than equivalent VMs for bursty workloads.



Reduced Ops Burden

No patching, capacity planning, or OS management. The provider handles all underlying infrastructure concerns.



Infinite Scale

Automatically handles 1 request or 1,000,000. AWS Lambda scales from 0 to 10,000 concurrent executions by default.



Event-Driven

Naturally reactive architecture. Triggers from HTTP, queues, databases, schedules, IoT events and more.



Speed to Market

Ship features, not infrastructure. Teams report 50–75% faster deployment cycles vs. traditional approaches.



Built-in HA

High availability and fault tolerance come standard. Functions run across multiple AZs automatically.

The Serverless Framework

Open-source CLI that turns `serverless.yml` > fully deployed cloud infrastructure

`serverless.yml`

Declare everything
in one config file

`sls deploy`

CLI packages code
& calls provider API

CloudFormation / ARM / Deployment Manager

IaC layer creates
all resources

Live Function + API + Resources

Running in the
cloud in seconds

Functions

Single-purpose units of business logic
— each maps to one cloud function
(Lambda, etc.)

Events

HTTP requests, S3 uploads,
DynamoDB streams, SQS messages,
cron schedules, WebSocket...

Resources

Databases, queues, buckets, IAM roles
— declared as
CloudFormation/Terraform resources

Plugins

1000+ community plugins: offline
emulation, TypeScript, webpack,
Datadog, Sentry...

The serverless.yml File Explained

```

service: my-rest-api
frameworkVersion: '3'

provider:
  name: aws
  runtime: nodejs20.x
  region: eu-west-1
  stage: ${opt:stage, 'dev'}
  iam:
    role:
      statements:
        - Effect: Allow

functions:
  getUser:
    handler: src/users.get
    timeout: 10
    events:
      - httpApi:
          path: /users/{id}
          method: get

resources:
  Resources:
    UsersTable:
      Type: AWS::DynamoDB::Table

```

service

Logical project name — used as prefix for all created AWS

resources provider

Target cloud + global settings (runtime, region, IAM, env vars, tags)

functions

Each entry = one Lambda. Handler points to file.exportedFunction

events

What triggers this function — HTTP, S3, SQS, schedule, WebSocket...

resources

CloudFormation resources (DynamoDB, S3, SQS, etc.) deployed alongside code

Multi-Cloud Support

Change provider: aws > azure — everything else stays the same.

AWS

Amazon Web Services

Primary — most complete support

Function:

Lambda

Triggers:

API Gateway, S3, SQS, EventBridge, DynamoDB
Streams

15-min timeout · 10k concurrent default · 1M free
req/mo

Azure

Microsoft Azure

Full support

Function:

Azure Functions

Triggers:

HTTP Trigger, Service Bus, Blob Storage, Event Grid

Consumption plan · Durable Functions support · .NET / JS
/ Python

GCP

Google Cloud Platform

Full support

Function:

Cloud Functions / Cloud Run

Triggers:

Pub/Sub, Firestore, Cloud Storage, HTTP

Cloud Run for containers · 2M free invocations/mo ·
Gen2 functions

Same CLI commands work everywhere: `sls deploy` / `sls invoke` / `sls logs` / `sls remove`

Deploy in 5 Steps

A complete function live on AWS from a blank terminal:

```
$ npm install -g serverless
```

Install the CLI

One-time global install. Requires Node.js ≥ 18 .

```
$ serverless create \
  --template aws-nodejs \
  --path my-service
```

Scaffold a project

Generates handler.js + serverless.yml with sensible defaults.

```
$ cd my-service && sls deploy
```

Deploy to AWS

Packages code > uploads to S3 > executes CloudFormation stack.

```
$ sls invoke -f hello \
  --data '{"name":"world"}'
```

Invoke remotely

Call the function directly. Returns response + logs.

```
$ sls remove
```

Teardown everything

Deletes the entire CloudFormation stack — no orphaned resources.

Community & Adoption

59K+

GitHub Stars

Top IaC tool

1M+

Weekly Downloads

npm installs / week

1,000+

Plugins

Community extensions

2015

Founded

9+ years of maturity



Why people love it

- Zero infra management — focus 100% on code
- Deploy to any major cloud with one CLI
- Large plugin ecosystem (offline, webpack, TS...)
- Infrastructure as Code baked in from day 1
- Active community, great docs, many examples



Known limitations

- YAML ties you to provider-specific APIs
- Cold starts (mitigated with Provisioned Concurrency)
- Max 15 min exec time (AWS Lambda hard limit)
- Debugging locally requires serverless-offline plugin
- v3>v4 had a paid model (now free for solo devs)

Serverless Framework vs Alternatives

Tool	Multi-Cloud	FaaS Focus	Learning Curve	IaC Built-in	Best For
Serverless Framework	Yes	Primary	Low	Yes	Rapid FaaS across clouds
AWS SAM	AWS only	Yes	Medium	Yes	AWS-native teams
Terraform	Yes	Partial Manual	High	Yes	Full infra, any cloud
Pulumi	Yes	Partial Manual	Medium	Yes	Code-first IaC
AWS CDK	AWS only	Yes	Medium	Yes	Complex AWS architectures
SST (Ion)	AWS only	Primary	Low	Yes	Full-stack AWS apps

Serverless Framework shines when you need FaaS across multiple clouds without deep IaC expertise.

Resources to Go Deeper



Official Docs

serverless.com/framework/docs

Full reference for YAML config, CLI commands, providers, plugins



Serverless Guru

serverlessguru.com

Advanced articles on CI/CD, multi-cloud patterns, performance tuning



GitHub — Examples Repo

github.com/serverless/examples

50+ real-world templates: REST API, GraphQL, ML, auth, scheduled jobs



Udemy Courses

udemy.com > 'Serverless Framework'

Practical video courses with hands-on labs — deploy real apps



Serverless Blog

serverless.com/blog

Case studies, best practices, provider comparisons, announcements



serverless-offline Plugin

github.com/dherault/serverless-offline

Emulate API Gateway & Lambda locally — essential for dev workflow

Key Takeaways



Serverless = event-driven compute, zero infra, pay-per-execution — a fundamental shift in cloud architecture.



Serverless Framework unifies AWS, Azure & GCP under one YAML file and CLI — write once, deploy anywhere.



Productivity win: from blank terminal to live HTTP endpoint in under 5 minutes, with full IaC.



Mature & widely adopted — 59K GitHub stars, 1M+ weekly downloads, trusted by startups and Fortune 500s.



Know the trade-offs: cold starts, 15-min exec limit, vendor-specific YAML, and stateless constraints.